

Bridging Search and Recommendation in Generative Retrieval: Does One Task Help the Other?

Gustavo Penha¹, Ali Vardasbi¹, Enrico Palumbo², Marco de Nadai³, Hugues Bouchard⁴

Spotify

¹Netherlands, ²Italy, ³Denmark, ⁴Spain

{gustavop,aliv,enricop,mdenadai,hb}@spotify.com

ABSTRACT

Generative retrieval for search and recommendation is a promising paradigm for retrieving items, offering an alternative to traditional methods that depend on external indexes and nearest-neighbor searches. Instead, generative models directly associate inputs with item IDs. Given the breakthroughs of Large Language Models (LLMs), these generative systems can play a crucial role in centralizing a variety of Information Retrieval (IR) tasks in a single model that performs tasks such as query understanding, retrieval, recommendation, explanation, re-ranking, and response generation. Despite the growing interest in such a unified generative approach for IR systems, the advantages of using a single, multi-task model over multiple specialized models are not well established in the literature. This paper investigates whether and when such a unified approach can outperform task-specific models in the IR tasks of search and recommendation, broadly co-existing in multiple industrial online platforms, such as Spotify, YouTube, and Netflix. Previous work shows that (1) the latent representations of items learned by generative recommenders are biased towards popularity, and (2) content-based and collaborative-filtering-based information can improve an item’s representations. Motivated by this, our study is guided by two hypotheses: [H1] the joint training regularizes the estimation of each item’s popularity, and [H2] the joint training regularizes the item’s latent representations, where search captures content-based aspects of an item and recommendation captures collaborative-filtering aspects. Our extensive experiments with both simulated and real-world data support both [H1] and [H2] as key contributors to the effectiveness improvements observed in the unified search and recommendation generative models over the single-task approaches.

KEYWORDS

Generative Retrieval, Generative Recommendation, Joint Search and Recommendation, Multi-task Learning

Table 1: Input and outputs for generative recommendation (Gen_R) and generative retrieval (Gen_S). The id strategy ϕ returns the token(s) that identify the item (IDs). The joint model (Gen_{R+S}) is trained with both datasets.

Generative recommendation (Gen_R)		
	Description	Example
Input	Tokens for history items	$[\phi(\text{item}_1), \phi(\text{item}_2)]$
Output	Tokens for target item	$\phi(\text{item}_3)$
Generative retrieval (Gen_S)		
	Description	Example
Input	Tokens for textual query	$\text{tokenize}(\text{"brazilian jazz"})$
Output	Tokens for relevant item	$\phi(\text{item}_3)$

ACM Reference Format:

Gustavo Penha¹, Ali Vardasbi¹, Enrico Palumbo², Marco de Nadai³, Hugues Bouchard⁴. 2024. Bridging Search and Recommendation in Generative Retrieval: Does One Task Help the Other?. In *18th ACM Conference on Recommender Systems (RecSys '24)*, October 14–18, 2024, Bari, Italy. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3640457.3688123>

1 INTRODUCTION

The recent breakthroughs of Large Language Models (LLMs) have significantly influenced the field of Information Retrieval (IR). In search, where the task is to rank a set of relevant documents for a query, LLMs can be employed to learn better textual representations in both sparse [8, 22, 33] and dense retrieval [1, 16, 19], re-rank items directly [35, 40, 41], and support evaluation [6, 11]. Similarly, in recommendation systems, where the objective is to rank a set of relevant items for a user, LLMs can improve user and item textual representations [27, 63], generate explanations for recommendations [49], and perform conversational recommendation [37, 60].

However, employing LLMs to retrieve items¹ for a given query or user is challenging. Architectures based on LLMs such as Cross-encoders [35, 40, 41] lack efficiency for large sets of items due to their self-attention mechanisms that process the tokens of the query and the tokens of the documents together. Although Bi-encoders [19] and Two-tower models [67] can employ LLMs for retrieval, they cannot leverage the attention across the query and document tokens [26]. Generative retrieval [4, 52] has been recently introduced to perform retrieval directly within a pre-trained LLM. This method learns to predict document unique identifiers (IDs) for input queries, bypassing the need for separate text-based encodings

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
RecSys '24, October 14–18, 2024, Bari, Italy

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0505-2/24/10...\$15.00
<https://doi.org/10.1145/3640457.3688123>

¹We use the terms *item* and *document* interchangeably in this paper to refer to the units of information that can be recommended and searched for.

of queries and documents. Thus, generative retrieval might be effectively applied in a multi-task learning framework [32] to develop a unified model composed of a single LLM, rather than multiple task-specific models. Research indicates that a single model addressing both search and recommendation could enhance the effectiveness of both tasks [69, 70]. However, such a multi-task approach remains under-explored in the generative IR literature.

This paper seeks to understand the circumstances under which a unified generative model for search and recommendation is beneficial for both tasks. To do so, we study how and when the search task can improve the effectiveness of the recommendation task and vice-versa. Table 1 shows examples of input and output for the recommendation task, in which a generative recommender generates item IDs based on past user interactions, and for the search task, in which a generative retrieval model generates item IDs for a given query. A joint generative model is trained for both tasks.

Previous work shows that (1) the latent representation of items, learned by generative models, are biased towards items popularity² [28], and (2) content-based and collaborative-filtering-based information can improve an item’s representations [36, 54, 68]. Based on those observations, we formulate two guiding hypotheses for our experiments: [H1] the joint training regularizes the estimation of each item’s popularity, and [H2] the joint training regularizes the item’s latent representations. Our main findings based on simulated and real-world data are:

- Improvements in the effectiveness of the joint generative model over the task-specific ones are evident in the simulated data for both hypotheses, provided that there is low KL divergence [21] between the popularity distributions of items in search and recommendation for [H1] and that the distribution of item co-occurrences aligns with the other task for [H2], thereby making the regularization effects helpful.
- The joint training of generative retrieval models for both recommendation and search proves more effective than the task-specific models for both tasks across three real-world datasets, showing an average increase of 16% in R@30. Our follow-up analyses suggest that the regularization effect on the item’s latent representation (i.e., [H2]) is the primary reason the predictions of the joint generative model differ from those of the task-specific models.

2 RELATED WORK

Generative Retrieval. Traditional information retrieval methods first encode all documents into a sparse or dense latent representation space and then find the closest document to the query at test time. In contrast, generative retrieval models [4, 24, 52, 61, 62, 65, 72, 74] learn to directly map queries to document IDs, enabling end-to-end pipelines within a single LLM. One of the first retrieval methods to use a Transformer-based language model (LM) is GENRE [4], which performs entity retrieval to directly output IDs for a query. GENRE predicts the entity name in an auto-regressive fashion, e.g. “Leonardo” → “Leonardo da” → “Leonardo da Vinci”, using constrained beam search. Generative retrieval has been popularized in IR by DSI [52], which uses a LM to predict relevant

documents for a given query, where documents are represented by IDs (e.g. doc_01) rather than its text—leading to fewer tokens representing each document and enables to control tokens that are shared across documents. Considering that we have weights tied to document tokens, generative retrieval requires each document to be explicitly linked to model weights in the LM head.

Tying documents to specific weights in the model requires documents to have been previously observed at training time to be predicted. For this reason, generative retrieval faces challenges of scalability and ingestion of new documents [20, 31]. Moreover, generative retrieval struggles to outperform and replace existing retrieval approaches when used for larger sets of items [39]. Recent studies indeed show that additional techniques are required to have competitive generative retrieval approaches [72, 73]. In this paper, we focus on the effects an additional retrieval recommendation task might have when employing multi-task learning. We expect our results to be agnostic to architectural choices and other explorations in literature aimed at improving the effectiveness and scalability of generative retrieval models.

Generative Recommendation. While search models retrieve documents based on queries, recommender systems retrieve items based on user’s past interactions. These systems may use textual metadata (content-based), historical interactions (collaborative filtering) or both. Traditional models such as the Two-tower model [67] learn embeddings for both users and items, and use neighbor search to retrieve a candidate set for recommendation, typically followed by a re-ranking step. In contrast, generative recommender systems can directly retrieve items from a collection for a given user in a single step. Early approaches relied only on user interactions using technologies ranging from LSTMs [14] to Transformers [17] to learn a mapping between users and items. More recently, LLMs have gained popularity for their ability to mix item IDs and natural language [7, 25]. A notable example is P5 [9], which combines text and IDs³ in a pre-trained P5 model to handle multiple recommendation tasks, such as sequential recommendation and rating prediction, within a multi-task setting⁴. While generative search and recommendation have become popular separated research directions [23] we study here the effect of learning both in the same model.

Joint Search and Recommendation. Search and recommendation tasks share similarities, both in terms of the fundamental problem [2]—to identify objects that satisfy users’ needs—and in the way models match queries or users (potentially with contextual information) with documents or items [64]. In some domains and platforms such as Spotify, YouTube, and Netflix, items can be accessed through textual search queries or recommendations based on a user’s historical interactions and current context. In such cases, a joint search and recommendation model could benefit from the signals coming from both ways to interact with items [48, 58].

Gong et al. [12] argued that in industrial applications, the data collected from either search or recommendation scenarios does not fully capture users’ intents, and shows that a multi-task learning

²Popularity of an item is measured by its number of interactions within the recommendation task, or the number of queries leading to it in the search task.

³One prompt in P5 could be for example *Given the purchase history list of user_15466: 4110 → 4467 → 4468 → 4472. Find the next item.*

⁴While P5 uses random IDs, more sophisticated content-based ways to create semantic IDs have also been proposed Hua et al. [15], Rajput et al. [44], paralleling semantic strategies used to represent document IDs for search.

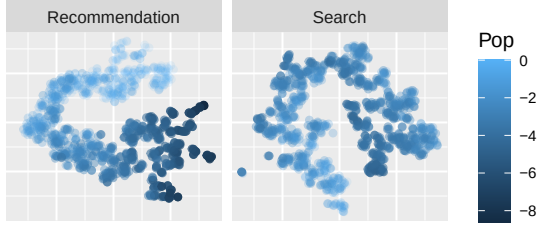


Figure 1: t-SNE [55] projection of the latent representations of a generative recommender (left) and generative search (right) for the ML dataset. The popularity of each item (darker if the item occurs more often in the respective recommendation and search datasets) is encoded in the embeddings of the learned item IDs in both generative retrieval models.

approach for seven IR tasks (e.g. Query-Item Relevance Prediction and Content Search Ranking) is more effective in an A/B test for the cold-start problem. Unlike personalized search systems [5], where user previous preferences are used to improve search and models that learn the transitions between search and recommendation surfaces [47, 66], we focus here on the problem of learning better item representations with both search and recommendation data. Literature has also hypothesized that sharing training data from both tasks enhances the item representations [69, 70, 75]. Although there is evidence for the improved effectiveness of joint learning over individual models, it remains unknown whether this applies to generative models and the underlying reasons each task could benefit from the other. In this paper, we analyze how search and recommendation tasks could benefit from each other in generative retrieval, investigating the learned popularity distribution of items and the shared item representations in generative models.

3 RESEARCH HYPOTHESES

Before defining our first hypothesis, let us discuss our motivation to investigate the popularity of items in generative retrieval. Previous work shows that item IDs in generative recommendation systems tend to exhibit a bias towards more popular items [28]. In Figure 1 we show this bias exists in both search⁵ and recommendation tasks using the latent representations of items (learned by generative retrieval models) in ML, a dataset derived from Movie-Lens 25M [13] (see Section 5). Considering the importance of item popularity in generative retrieval, our first hypothesis, [H1], is the following:

Hypothesis 1 (Popularity). A joint model for search and recommendation regularizes the estimation of each item’s popularity.

To give a concrete example, let us analyze the first row of Table 2, which shows our motivational examples to illustrate our main hypotheses. Consider a recommendation dataset comprising four items with training popularity distribution of $Pop_R^{Train} = \{20\%, 0\%, 50\%, 10\%\}$ (third column of Table 2), indicating for example that the first item appears on 20% of the instances. Let’s assume the train distribution differs from the distribution at test time $Pop_R^{Test} = \{10\%, 30\%, 40\%, 10\%\}$. This represents a shift from training to test conditions [42]. Suppose that the search data’s training set distribution is $Pop_S^{Train} = \{5\%, 40\%, 50\%, 5\%\}$, for example

because users behave differently across surfaces. A model trained on both datasets (see column R+S from Table 2 for the first row) would retrieve the second item more frequently compared to a model trained solely on Pop_R^{Train} , leading to better effectiveness on its own shifted test set. For the regularization coming from H1 to increase the effectiveness of the joint model in real-world applications, we hypothesize that the popularity distributions between search and recommendation have to be related but not equal, e.g. low KL Divergence, and the individual datasets have to be insufficient for learning a robust estimation of the test set popularity of each item (e.g. a shift between train and test distributions exists).

Next, we discuss our second hypothesis motivated by previous work on content-based and collaborative-based hybrids [36, 54, 68] and on joint search and recommendation models [69, 70]:

Hypothesis 2 (Latent representation). A joint model for search and recommendation regularizes the item’s latent representations, meaning that the patterns learned to position items across the latent space in one task can beneficially influence the other.

For example, consider the $S \rightarrow R$ task (Search to Recommendation), shown in the second row of Table 2 where recommendation is the target task⁶. Suppose a scenario in which R aims to predict *item₁* from user u_1 who previously interacted with $\{item_3 \rightarrow item_2\}$. The recommendation model might struggle to make this prediction if the pair $\{item_3, item_1\}$ and $\{item_2, item_1\}$ have rarely (or never) been observed in the training dataset D_{rec}^{Train} . However, the search task could fill this data scarcity if D_{search}^{Train} contains semantically similar queries that lead to those three items. Consider for example that, *item₁* is relevant for the queries “*medieval setting*”, “*historical fiction*”, and *item₂* is relevant for the same set of queries. Consequently, the joint model may learn an embedding space where $\{item_1, item_2\}$ are closer, leading to correctly predicting *item₁* as a likely next item to *item₂*.

Let’s now examine an example in the reverse direction $R \rightarrow S$ (third row in Table 2), where the regularization coming from the recommendation objective might help the search task. Consider a test query q_3 “*ancient history*” where the relevant documents are $\{item_3, item_2, item_1\}$ and that a learned retrieval model correctly retrieves $\{item_1, item_3\}$ (they are close to the query in the semantic space). If the pairs $\{item_1, item_2\}$ and $\{item_2, item_3\}$ are prevalent in the D_{rec}^{Train} , the joint model will push *item₃* closer to $\{item_1, item_2\}$, and consequently to the query, enhancing its retrieval accuracy.

4 JOINT SEARCH AND RECOMMENDATION GENERATIVE MODEL

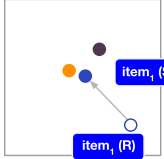
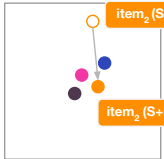
In generative retrieval, a function ϕ maps each item in the collection to its respective identifier, which might contain one or more tokens. The vocabulary of the underlying pre-trained LM is composed of the vocabulary tokens that represent textual natural language and of the tokens used to represent the items in the collection. We use atomic IDs⁷ for ϕ , and thus we have one additional token per item in the vocabulary. Generative models are trained auto-regressively

⁶We refer to *target* task as the one we are currently evaluating the joint model on.

⁷Future work may explore replacing these atomic IDs with semantic IDs [15, 44, 52], based on content or collaborative embeddings, to scale to a larger set of items.

⁵Note that many search datasets do not have unequal distributions of queries to documents, such as MSMarco [34] and BEIR [53].

Table 2: Motivational examples for our hypothesis. For each row of this table, we have examples of search and recommendation datasets, and the expected effect of training the generative retrieval model on both objectives. Each example motivates a hypothesis we present in the paper. For the second hypothesis, we consider one example for each direction: $S \rightarrow R$ indicates that the target task is recommendation, and thus we expect the search example to improve the joint model effectiveness. Similarly, $R \rightarrow S$ indicates that the target task is search, and thus we expect the recommendation examples to improve effectiveness compared to a model trained only on search. The images show how the representation of the items could change when going from the task-specific models, $item_i(S)$ or $item_i(R)$ to the joint model $item_i(S+R)$.

Hypotheses	Search (S) example	Recommendation (R) example	Effect R+S joint training
[H1] Popularity	$Pop_R^{Train} = \{5\%, 40\%, 50\%, 5\%\}$	$Pop_S^{Train} = \{20\%, 0\%, 50\%, 10\%\}$	$Pop_{S+R}^{Train} = \{12.5\%, 20\%, 50\%, 7.5\%\}$ gets closer to $Pop_R^{Test} = \{10\%, 30\%, 40\%, 10\%\}$
[H2] Latent rep. $S \rightarrow R$	q_1 : “medieval setting” q_1 relevant items: $\{item_1, item_2\}$ q_2 : “historical fiction” q_2 relevant items: $\{item_1, item_2\}$	$u_1: \{item_3 \rightarrow item_2 \rightarrow item_1\}$	 The embedding of $item_1$ using S+R becomes closer to $item_2$.
[H2] Latent rep. $R \rightarrow S$	q_3 : “ancient history” q_3 relevant items: $\{item_1, item_2, item_3\}$	$u_2: \{item_3 \rightarrow item_2\}$ $u_3: \{item_1 \rightarrow item_2\}$	 The embedding of $item_2$ using S+R becomes closer to $item_1$ and $item_3$ and consequently to the query q_3.

with teacher forcing, employing cross-entropy loss between the predicted ID tokens and the ground truth ID tokens. To perform retrieval with generative retrieval, beam search is performed, returning the top K valid item IDs.

Let $\mathcal{D}_S = \{(Q_i, \{item_1, item_2, \dots, item_k\})\}_{i=1}^N$ be a search dataset comprised of relevance labels for queries, where Q is the query and $\{item_1, item_k, \dots, item_k\}$ are the items that are relevant for this query. To train a generative model on this dataset, each query turns into k input-output pairs of the following format:

$$[(Q, \phi(item_1)), \dots, (Q, \phi(item_k))]$$

We refer to a generative model trained on $\mathcal{D}_S$ as Gen_S . Let $\mathcal{D}_R = \{(U_i = \{item_1, item_2, \dots, item_{t-1}\}, item_t)\}_{i=1}^M$ be a recommendation dataset comprised of user interactions split into history and target pairs, where the history are the previous interactions of the user sorted by time, and the target item is his last interacted item. To train a generative model on this dataset, each user turns into one pair of the following format:

$$(\underbrace{\text{concat}([\phi(item_1), \phi(item_2), \dots, \phi(item_{t-1})])}_{\text{history}}, \underbrace{\phi(item_t)}_{\text{target}})$$

where $\text{concat}(\cdot)$ is the concatenation of the item IDs with a space token. We refer to a generative model trained on $\mathcal{D}_R$ as Gen_R . We refer to a single generative retrieval model on both $\mathcal{D}_R$ and $\mathcal{D}_S$ as Gen_{R+S} . See Table 1 for examples.

5 EXPERIMENTAL SETUP

In this section, we describe the datasets, the evaluation protocol, and the implementation details we use to analyze the effect of joint training of search and recommendation for generative retrieval.

Simulated datasets. We create three simulated datasets⁸ to analyze our hypotheses [H1] and [H2]. For each dataset, we hold the target task data constant and alter the data of the other task based on the specific parameter we want to analyze:

- (i) **SIM1:** This dataset is used to test hypothesis [H1]. We modify the *KL divergence* between the popularity distributions of items to examine how differences in item popularity affect the model’s performance. Specifically, we begin by defining a popularity distribution for each item based on Zipf’s law [10] distribution. Then, we incrementally shuffle the item probability distribution in the search dataset while maintaining a constant distribution in the recommendation dataset. This method introduces increasing divergence between the two popularity distributions. The only learnable signal in both datasets is the popularity bias, allowing us to analyze the impact of one task on the other in a controlled manner, according to how different the popularity distributions are;
- (ii) **SIM2:** We vary the percentage of queries that lead to the items of the user history which match the queries, i.e. are

⁸The simulated datasets and their statistics are available at: <https://anonymous.4open.science/r/simulated-deatasets-recsys24-DB90/stats.md>

the same, that lead to the target items (*% of q . match*) to test [H2] in the $S \rightarrow R$ direction. We simulate a recommendation dataset composed of five clusters, each containing six items that frequently co-occur in user interaction data. User histories are created by randomly sampling an initial item and pairing it with a second item that comes from the same cluster. We do this until the number of desired interactions per user is reached. This setup is designed to encode the co-occurrences of items within specific clusters as the learnable information in the dataset. When generating the search data, we modify how many queries are distributed for items in the same clusters. For example, with *% of q . match* at 100% all queries for items within a cluster are the same, whereas, at 50% overlap, only half of the queries are the same, with the remainder being randomly selected;

- (iii) **SIM3**: This dataset tests hypothesis [H2] in the $R \rightarrow S$ direction by altering the percentage of item pairs in recommendation data that also appear together in relevant query lists (*% pairs in $qrels$*). We simulate a search scenario using a subset of ten randomly selected topics from TREC-DL22 [3], providing realistic text for the queries. For each topic, we generate five paraphrased queries using OpenAI GPT-4. One of the paraphrases is designated as the test query, while the remaining four serve as training instances. The recommendation dataset is constructed with varying percentages of appearances of recommendation item pairs in relevant groups of documents (that we refer to as *% pairs in $qrels$*). For example, consider a user with the following interactions $\{item_1, item_2, item_3\}$ and a search dataset with one query to relevance list $(q, \{item_2, item_3\})$. In this example, the percentage of relevant pairs is 33.33% as, out of the three pairs coming from all the combinations of items from the user interactions ($\{item_1, item_2\}$, $\{item_1, item_3\}$, and $\{item_2, item_3\}$), only one pair ($\{item_2, item_3\}$) appears in a set of relevant documents for a query.

For both recommendation and search tasks, we generate datasets of comparable sizes of training instances. Each model is run five times, and we report the average recall (R@10) values to ensure statistical reliability. We describe in more detail the sampling procedure of each dataset when discussing the results of the simulated datasets.

Real-world datasets. Then, we use three real-world datasets to evaluate the effectiveness of the joint search and recommendation generative model. These datasets are detailed in Table 3. (i) **ML**: Derived from the MovieLens 25M [13] dataset, employing user interactions for recommendations, and genres, tags, and genome-tags [56] as search queries. (ii) **MPD**: Constructed using the Million Playlist Dataset [71]⁹, where playlists serve as the recommendation dataset, and unique playlist titles are used as search queries. In both ML and MPD datasets, we ensure to have just items that appear in both datasets and that all items in the test set appear in the training set. In addition to these public datasets, we created (iii) **Podcasts** dataset to provide an even more realistic assessment of joint search and recommendation effectiveness. This dataset consists of a sample of real log queries and user interactions from Spotify (from 2023-07

to 2023-10), with podcast shows as units of retrieval. This dataset features 84% of items from the search in the recommendation data, and 30% of the items in the recommendation data are found in the search data, illustrating how behaviour can vary across different surfaces and samples of datasets. We filtered the search dataset in order to contain mostly broad queries that lead to successful search interactions, as the logs have a high percentage of narrow, known entity queries [38].

Evaluation. In the recommendation task (R), we use the user history to predict the last item (the target) in an ordered sequence of t items. During training, we construct the user’s history up to $(t-3)$ interactions and simulate additional training instances by sequentially predicting the last item and removing it from the sequence until only two items remain. The test data observes all interactions up to $t-1$ to predict the item at position t . The validation test predicts the item at position $t-1$ from the previous ones, while the training data does not see the last two interactions for each user. For the search task (S), we maintain distinct sets of queries for training, validation, and testing. Given our focus on the retrieval task, we rely on recall metrics: R@10 for the simulated datasets¹⁰ and R@30 for the, larger, real-world datasets. We assess the statistical significance of our results using paired Students’ t -tests with a 95% confidence interval.

Implementation Details. We use Huggingface’s implementation of T5 [43] (*google/flan-t5-base*) and train all generative models for 5 epochs with a learning rate of 0.002 and a batch size of 128. We use AdamW [29] with a weight decay of 0.01. To increase the number of distinct items retrieved for all generative retrieval models we resort to a diversified beam search approach [57]. We use a diversified beam search procedure with a diversity penalty of 0.25. The number of groups equals half the number of desired return items.

The models used for reference for the search task are BM25 [46] (Pyterrier [30] implementation with default hyperparameters) and a Bi-encoder [50] (SentenceTransformers [45] implementation using pre-trained *sentence-transformers/all-mpnet-base-v2*). The recommendation models used for reference for the recommendation task are SASRec [18] and BERT4Rec [51] using Recbole implementations [76], and DiffRec [59] using the author’s github code. Although such neural baselines could be referred to as generative recommendation approaches, they do not leverage natural language text tokens, and thus cannot handle the search dataset. Pop_R returns the most frequent items according to the recommendation training data and Pop_S does so according to search.

6 RESULTS FOR SIMULATED DATASETS

This section discusses the results from the simulated datasets. We begin by analyzing the item popularity under [H1] with the first simulated dataset (SIM1), followed by exploring the regularization effect on item representation for [H2] with SIM2 and SIM3.

6.1 SIM1: Item Popularity

We assess [H1] by training generative retrieval models on the datasets from the first simulation, each with varying degrees of KL divergence between item popularity distributions. Figure 2 provides

⁹<https://research.atspotify.com/2020/09/the-million-playlist-dataset-remastered/>

¹⁰We used a cutoff of 10 due to the small size of the simulated datasets (<100 entities).

Table 3: Statistics of the real-world datasets. Avg. rel. per query indicates the average number of relevant documents per query. The Gini index measures how concentrated the items are in the training dataset in terms of queries and users leading to them (higher values mean higher concentration). KSdist is the Kolmogorov Distance between the two distributions of popularities and KLD is the Kullback–Leibler divergence between them.

	Recommendation				Search				Popularity distributions comparison		
	# items	# users	Density	Gini index	# items	# queries	Avg. rel. per query	Gini index	KSdist(S,R)	KLD(S R)	KLD(R S)
ML	20k	163k	0.00750	0.74	20k	55k	6.86	0.50	0.42	0.91	0.81
MPD	21k	85k	0.00055	0.52	21k	28k	36.44	0.58	0.22	0.14	0.15
Podcasts	43k	211k	0.00030	0.73	15k	4k	10.27	0.56	0.46	1.49	1.55

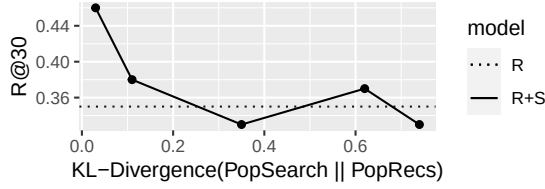


Figure 2: Results of SIM1 for [H1].

evidence supporting [H1]. It suggests that learning for both tasks within a single generative model regularizes the learned popularity distribution of the items. However, we note here that there is no shift between train and test popularity distributions in this simulation. Additionally, the only signal in the simulated data is the popularity of each entity diverging from real world datasets where entities that are similar co-occur more frequently.

6.2 SIM2: Item Representation ($S \rightarrow R$)

This section assesses our second hypothesis [H2] by focusing on recommendation as the target task. The results from simulation SIM2, presented in Table 4, show that a mismatch in queries across the clusters negatively affects the regularization, decreasing the effectiveness of Gen_{R+S} compared to Gen_R . However, increasing the number of matching queries within clusters enhances the performance relative to the baseline. This finding supports [H2], suggesting that the regularization can be beneficial particularly when there is a high degree of similarity between the search and recommendation data in terms of item co-occurrences (% of q . match). To visually assess the impact of joint training, we compare, in Figure 3, the item embeddings obtained from a recommendation-only model with the ones obtained by a joint model that incorporates both search (at 100% of q . match) and recommendation data. We observe that the joint model effectively groups items according to their underlying clusters compared to the model trained solely on recommendation.

6.3 SIM3: Item Representation ($R \rightarrow S$)

In this section, we validate our second hypothesis by focusing on search as the target task. Table 4 shows the results for simulation SIM3. They indicate that the joint model performs more effectively than the model trained solely on the recommendation data when the % pairs in $qrels$ exceeds a certain threshold. This finding supports [H2], suggesting that the multi-task learning objective can serve as an effective regularizer for item representations. This regularization is particularly beneficial when there is a high degree of

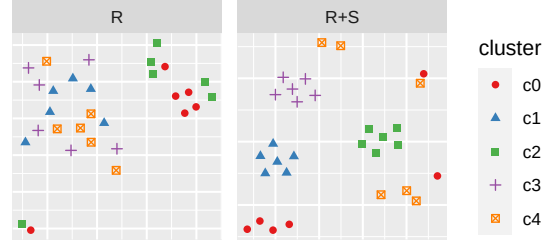


Figure 3: t-NSE [55] reduction of the embeddings of models learned for SIM2.

Table 4: Results of SIM2 and SIM3 for [H2]. ‡ indicates statistical significance against Gen_R (SIM2 results) and Gen_S (SIM3 results) using paired t-tests.

Sample	Model	(SIM2) $S \rightarrow R$		Model	(SIM3) $R \rightarrow S$	
		% of q . match	R@10		% pairs in $qrels$	R@10
100%	Gen_R	-	0.907	Gen_S	-	0.580
	Gen_{R+S}	0%	0.842	Gen_{R+S}	0%	0.240
		25%	0.880		25%	0.280
		50%	0.938‡		50%	0.280
		75%	0.944‡		75%	0.460
		100%	0.985‡		100%	0.640‡
65%	Gen_R	-	0.717	Gen_S	-	0.320
	Gen_{R+S}	0%	0.732	Gen_{R+S}	0%	0.200
		25%	0.752		25%	0.260
		50%	0.708		50%	0.220
		75%	0.903‡		75%	0.400‡
		100%	0.897‡		100%	0.380‡

similarity between the co-occurrences of items in the search and recommendation data.

7 RESULTS FOR REAL-WORLD DATASETS

In this section, we present the results from real-world data in a question-and-answer format to provide a structured understanding of the insights derived from the datasets. Table 5 shows the main results for the three real-world datasets (Head indicates the effectiveness for the top 1% most popular items in the train set, where Torso is the remaining set of items.). Note that Table 5 also

contains baselines (Pop_R, Pop_S, SASRec [18], BERT4Rec [51], and DiffRec [59] for the recommendation task and BM25 [46] and Bi-encoder [50] for the search task) as a reference. The aim of this research is not to provide a new generative model that is more effective than the non-generative counterparts but to bridge the gap of understanding when and why jointly training on search and recommendation benefits a *generative* model.

What is the effectiveness of the joint model compared to the task-specific models? In most scenarios, the joint model is more effective than the task-specific models for search and recommendation tasks. One exception is observed in the MPD dataset, where Gen_{R+S} only outperforms the recommendation-specific model (Gen_R) for Head items (R@30 0.333). These results underscore that the search and recommendation tasks mutually benefit each other in generative retrieval, with an average increase of 16% in R@30.

What are the discrepancies between task-specific and joint model predictions? Motivated by [H1], we explore the popularity of items retrieved by the generative models. Specifically, we compute the popularity increase of the items predicted when transitioning from a task-specific model to a joint model. For example, if the target task is recommendation, we measure the increase in popularity according to the search data of the retrieved items by Gen_{R+S} compared to the retrieved items by Gen_R. The results, shown in Table 6, indicate that joint training of the model does not always lead to an increase in the average popularity of predicted items (according to the new task we add). This could indicate that the effect on the item’s latent representation coming from [H2] is entangled and thus interfering with the popularity distribution.

Building on [H2], we examine the effect of regularization on items’ representations within the joint model. Here, we focus on the differences between the predictions of the task-specific and joint models. We measure two metrics: one for the recommendation task and another for the search task.

For the recommendation task, for each predicted list from Gen_{R+S} that is different than Gen_R, given the ground-truth target item $item_t$ and the history $\{item_1, \dots, item_{t-1}\}$, we count all the queries that have both $\{item_i, item_t\}$ ($i < t$) in their relevance set ($\Delta \# \text{ matches of history} \rightarrow \text{target queries}$). For example, if the user history includes $\{item_1, item_2\}$ and the target is $item_3$, and in the search data there are respectively 2 and 3 unique queries that have the items $\{item_1, item_3\}$ and $\{item_2, item_3\}$ in their corresponding relevant set, this value would be 5.

The second metric, $\Delta \text{ item pairs from } R \text{ in rel pairs for queries}$, applicable to the search task (second column of Table 7), calculates the average number of co-occurrences of the pairs of relevant items for a given query in the recommendation data. For example, if the relevant items for a query are $\{item_1, item_3, item_5\}$ and they co-occur in the recommendation data as follows: $\{item_1, item_3\} : 3$, $\{item_1, item_5\} : 4$, $\{item_3, item_5\} : 2$, the average score for the query would be $(3 + 4 + 2)/3$. The findings, detailed in Table 7, reveal when the joint model predicts different items than the task-specific models, both metrics we defined increase significantly. This indicates that the regularization of items’ latent representation significantly alters the joint model’s predictions, providing strong support for our second hypothesis.

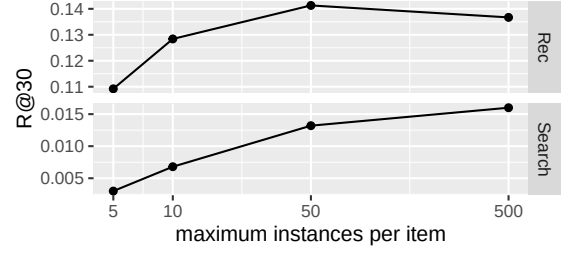


Figure 4: Effectiveness of Gen_{R+S} for the Podcasts dataset when increasing the number of training instances per item allowed from the added task.

What is the effect of removing the popularity information from the added task? Although disentangling the effects of both hypotheses from the real-world datasets is challenging, we attempt to remove popularity bias by limiting the number of training instances per item. By restricting the training data to a fixed number of instances per item we effectively remove the popularity bias in the dataset, although we also modify the item representation and co-occurrences for frequently occurring items. The results displayed in Figure 4 show a decrease in effectiveness under these conditions, supporting [H1]—that the regularization effect on the learned popularity distribution influences the gains from the joint model.

What is the effect of the redundancy of co-occurring items across the search and recommendation datasets? To further understand when the joint model is more effective, we split the test instances based on redundant and non-redundant pairs of items. We define a redundant pair of items if they appear both in the search training data, as relevant for a single query, and in the recommendation training data, as part of the same user history of interactions. A non-redundant pair does not appear in the training set of the target task but appears in the other task training data. While redundant pairs can reinforce useful relationships between pairs, the non-redundant pairs can fill missing patterns between the items that the original task dataset did not have.

The statistics related to the two types of pairs for the Podcasts dataset in Table 8 show that the number of pairs originating from the search dataset, which were not present in the recommendation training data, is relatively low (only 56 non-redundant pairs). This suggests that the observed gains in the Podcasts dataset for the recommendation task are likely not due to learning new relationships between pairs (that were not present in the recommendation data) but rather due to reinforcing the existing ones (31% improvement for redundant pairs). Conversely, for the search task, we do observe that the gains of the joint model over the task-specific one are higher for non-redundant pairs. This indicates that the regularization effect of [H2] can be beneficial for both redundant and non-redundant pairs of items.

8 CONCLUSION

In this paper, we have explored the impact of multi-task learning in generative retrieval models, specifically focusing on the search and recommendation tasks. We investigated two hypotheses that might explain the effectiveness improvements of multi-task learning for search and recommendation. The first hypothesis is that a joint

Table 5: R@30 for the joint generative model (Gen_{R+S}) compared to the task-specific models (Gen_S and Gen_R), and the respective percentual improvement. Bold indicates the highest score between the joint generative model and the task-specific generative model, and the superscript $\ddagger$ indicates statistical significance against the task-specific model using paired t-tests. Head indicates the effectiveness for the top 1% most popular items in the train set, where Torso is the remaining set of items.

	Recommendation								
	ML			MPD			Podcasts		
	All	Head	Torso	All	Head	Torso	All	Head	Torso
Generative retrieval methods									
Gen_R	0.103	0.267	0.057	0.067	0.269	0.043	0.112	0.334	0.018
Gen_{R+S}	0.119[‡]	0.307[‡]	0.066[‡]	0.055	0.333[‡]	0.021	0.149[‡]	0.345[‡]	0.067[‡]
<i>improv.</i>	(+16%)	(+15%)	(+16%)	(−18%)	(+24%)	(−50%)	(+33%)	(+3%)	(+262%)
Popularity and neural-based methods (for reference)									
Pop_R	0.069	0.312	0.000	0.028	0.259	0.000	0.073	0.247	0.000
Pop_S	0.062	0.264	0.005	0.026	0.241	0.000	0.029	0.091	0.003
SASRec	0.207	0.356	0.164	0.231	0.462	0.205	0.256	0.546	0.134
BERT4Rec	0.247	0.401	0.203	0.151	0.460	0.115	0.260	0.518	0.151
DiffRec	0.211	0.528	0.121	0.215	0.335	0.201	0.280	0.442	0.212
Search									
	ML			MPD			Podcasts		
	All	Head	Torso	All	Head	Torso	All	Head	Torso
	All	Head	Torso	All	Head	Torso	All	Head	Torso
Generative retrieval methods									
Gen_S	0.020	0.053	0.000	0.028	0.040	0.002	0.080	0.148	0.004
Gen_{R+S}	0.023[‡]	0.060[‡]	0.001	0.033[‡]	0.044[‡]	0.009[‡]	0.106[‡]	0.171[‡]	0.034[‡]
<i>improv.</i>	(+16%)	(+12%)	(−)	(+18%)	(+11%)	(+373%)	(+33%)	(+16%)	(+855%)
Sparse and dense methods (for reference)									
BM25	0.091	0.060	0.109	0.032	0.017	0.065	0.304	0.237	0.383
Bi-encoder	0.081	0.062	0.092	0.032	0.017	0.065	0.269	0.212	0.336

Table 6: Analysis of model predictions for [H1].

	Recommendation	Search
	$\Delta \text{Pop}_S \text{ Gen}_R \rightarrow \text{Gen}_{R+S}$	$\Delta \text{Pop}_R \text{ Gen}_S \rightarrow \text{Gen}_{R+S}$
ML	−2.72%	+33.23%
MPD	+0.92%	−7.36 %
Podcasts	−17.46%	−13.33%

Table 7: Analysis of model predictions for [H2].

	Recommendation	Search
	$\Delta \# \text{ matches of history} \rightarrow \text{target queries}$	$\Delta \text{ item pairs from } R \text{ in rel pairs for queries}$
ML	+183.09%	+136.47%
MPD	+210.09%	+213.01%
Podcasts	+170.92%	+314.21%

model can achieve a more robust estimation of each item’s popularity, i.e. its frequency of appearance in user interactions or queries. The second hypothesis is that the joint learning objective provides a beneficial regularization effect on the item latent representations.

Table 8: Statistics related to the redundancy of item pairs across tasks for the Podcasts dataset and their effect on finding relevant items.

	Recommendation	Search
Redundant # pairs	1933	64146
Improv. of Gen_{R+S} for redundant	31.81%	61.39%
Non-redundant # pairs	56	11222
Improv. of Gen_{R+S} for non-redun.	0.00%	86.76%

Using simulated datasets, we provided evidence supporting both hypotheses. We also identify scenarios where improvements might not manifest: e.g. when there is a high divergence between the items’ popularity distributions or when the items’ co-occurrences do not align across tasks. Additionally, we used three different real-world datasets to show that the multi-task learned generative retrieval model yields effectiveness improvements over the task-specific models, with an average increase of +16% in R@30. Our analysis detailed the conditions under which the joint model outperforms task-specific models, examining changes in predictions, the impact

of eliminating popularity bias in training data and analyzing the effect of item pair redundancy across tasks.

We believe this research marks a significant step towards developing unified LLMs for a broad range of IR tasks, shedding light on the specific contributions of search and recommendation tasks for generative retrieval. For future research, we plan to explore the effect of integrating additional tasks, such as generating explanations, within a unified multi-task learned LLM for IR and to examine ID strategies (how to represent each item based on a set of tokens) in the multi-task learning framework.

REFERENCES

- [1] Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. *arXiv preprint arXiv:2404.05961* (2024).
- [2] Nicholas J Belkin and W Bruce Croft. 1992. Information filtering and information retrieval: Two sides of the same coin? *Commun. ACM* 35, 12 (1992), 29–38.
- [3] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, Jimmy Lin, Ellen M. Voorhees, and Ian Soboroff. 2023. Overview of the TREC 2022 deep learning track. In *Text REtrieval Conference (TREC)*. NIST, TREC. <https://www.microsoft.com/en-us/research/publication/overview-of-the-trec-2022-deep-learning-track/>
- [4] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2020. Autoregressive entity retrieval. *arXiv preprint arXiv:2010.00904* (2020).
- [5] Zhicheng Dou, Ruihua Song, and Ji-Rong Wen. 2007. A large-scale evaluation and analysis of personalized search strategies. In *Proceedings of the 16th international conference on World Wide Web*. 581–590.
- [6] Guglielmo Faggioli, Laura Dietz, Charles LA Clarke, Gianluca Demartini, Matthias Hagen, Claudia Hauff, Noriko Kando, Evangelos Kanoulas, Martin Potthast, Benno Stein, et al. 2023. Perspectives on large language models for relevance judgment. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*. 39–50.
- [7] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).
- [8] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE v2: Sparse lexical and expansion model for information retrieval. *arXiv preprint arXiv:2109.10086* (2021).
- [9] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*. 299–315.
- [10] K George. [n. d.]. Zipf. 1935. The psycho-biology of language.
- [11] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences* 120, 30 (2023), e2305016120.
- [12] Yuqi Gong, Xichen Ding, Yehui Su, Kaiming Shen, Zhongyi Liu, and Guannan Zhang. 2023. An Unified Search and Recommendation Foundation Model for Cold-Start Scenario. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4595–4601.
- [13] F Maxwell Harper and Joseph A Konstan. 2015. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)* 5, 4 (2015), 1–19.
- [14] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).
- [15] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to Index Item IDs for Recommendation Foundation Models. *arXiv preprint arXiv:2305.06569* (2023).
- [16] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118* (2021).
- [17] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [18] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. *arXiv:1808.09781* [cs.IR]
- [19] Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906* (2020).
- [20] Varsha Kishore, Chao Wan, Justin Lovelace, Yoav Artzi, and Kilian Q Weinberger. 2023. IncDSI: Incrementally Updatable Document Retrieval. (2023).
- [21] Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics* 22, 1 (1951), 79–86.
- [22] Carlos Lassance, Hervé Déjean, Thibault Formal, and Stéphane Clinchant. 2024. SPLADE-v3: New baselines for SPLADE. *arXiv preprint arXiv:2403.06789* (2024).
- [23] Yongqi Li, Xinyu Lin, Wenjie Wang, Fuli Feng, Liang Pang, Wenjie Li, Liqiang Nie, Xiangnan He, and Tat-Seng Chua. 2024. A Survey of Generative Search and Recommendation in the Era of Large Language Models. *arXiv:2404.16924* [cs.IR]
- [24] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2023. Multiview Identifiers Enhanced Generative Retrieval. *arXiv preprint arXiv:2305.16675* (2023).
- [25] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al. 2023. How can recommender systems benefit from large language models: A survey. *arXiv preprint arXiv:2306.05817* (2023).
- [26] Jimmy Lin, Rodrigo Nogueira, and Andrew Yates. 2022. *Pretrained transformers for text ranking: Bert and beyond*. Springer Nature.
- [27] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2024. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 452–461.
- [28] Zhenghao Liu, Sen Mei, Chenyan Xiong, Xiaohua Li, Shi Yu, Zhiyuan Liu, Yu Gu, and Ge Yu. 2023. Text Matching Improves Sequential Recommendation by Reducing Popularity Biases. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1534–1544.
- [29] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [30] Craig Macdonald, Nicola Tonellotto, Sean MacAvaney, and Iadh Ounis. 2021. PyTerrier: Declarative experimentation in Python from BM25 to dense retrieval. , 4526–4533 pages.
- [31] Sanket Vaibhav Mehta, Jai Gupta, Yi Tay, Mostafa Dehghani, Vinh Q Tran, Jinfeng Rao, Marc Najork, Emma Strubell, and Donald Metzler. 2022. DSI++: Updating Transformer Memory with New Documents. *arXiv preprint arXiv:2212.09744* (2022).
- [32] Donald Metzler, Yi Tay, Dara Bahri, and Marc Najork. 2021. Rethinking search: making domain experts out of dilettantes. In *Acm sigir forum*, Vol. 55. ACM New York, NY, USA, 1–27.
- [33] Thong Nguyen, Sean MacAvaney, and Andrew Yates. 2023. A Unified Framework for Learned Sparse Retrieval. In *European Conference on Information Retrieval*. Springer, 101–116.
- [34] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. Ms marco: A human-generated machine reading comprehension dataset. (2016).
- [35] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085* (2019).
- [36] Govindarajan Parthasarathy and Shanmugam Sathya Devi. 2023. Hybrid Recommendation System Based on Collaborative and Content-Based Filtering. *Cybernetics and Systems* 54, 4 (2023), 432–453.
- [37] Gustavo Penha and Claudia Hauff. 2020. What does bert know about books, movies and music? probing bert for conversational recommendation. In *Proceedings of the 14th ACM conference on recommender systems*. 388–397.
- [38] Gustavo Penha, Enrico Palumbo, Maryam Aziz, Alice Wang, and Hugues Bouchard. 2023. Improving content retrievability in search with controllable query generation. In *Proceedings of the ACM Web Conference 2023*. 3182–3192.
- [39] Ronak Pradeep, Kai Hui, Jai Gupta, Adam D Lelkes, Honglei Zhuang, Jimmy Lin, Donald Metzler, and Vinh Q Tran. 2023. How Does Generative Retrieval Scale to Millions of Passages? *arXiv preprint arXiv:2305.11841* (2023).
- [40] Ronak Pradeep, Sahel Sharifmoghadam, and Jimmy Lin. 2023. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088* (2023).
- [41] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, et al. 2023. Large language models are effective text rankers with pairwise ranking prompting. *arXiv preprint arXiv:2306.17563* (2023).
- [42] Joaquin Quiñero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D Lawrence. 2022. *Dataset shift in machine learning*. Mit Press.
- [43] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [44] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, et al. 2023. Recommender Systems with Generative Retrieval. *arXiv preprint arXiv:2305.05065* (2023).
- [45] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. <https://arxiv.org/abs/1908.10084>
- [46] Stephen E Robertson and Steve Walker. 1994. Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval. In *SIGIR '94*.

- Springer, 232–241.
- [47] Teng Shi, Zihua Si, Jun Xu, Xiao Zhang, Xiaoxue Zang, Kai Zheng, Dewei Leng, Yanan Niu, and Yang Song. 2024. UniSAR: Modeling User Transition Behaviors between Search and Recommendation. *arXiv preprint arXiv:2404.09520* (2024).
- [48] Zihua Si, Zhongxiang Sun, Xiao Zhang, Jun Xu, Xiaoxue Zang, Yang Song, Kun Gai, and Ji-Rong Wen. 2023. When Search Meets Recommendation: Learning Disentangled Search Representation for Recommendation. *arXiv preprint arXiv:2305.10822* (2023).
- [49] Ítallo Silva, Leandro Marinho, Alan Said, and Martijn C Willemsen. 2024. Leveraging ChatGPT for Automated Human-centered Explanations in Recommender Systems. In *Proceedings of the 29th International Conference on Intelligent User Interfaces*. 597–608.
- [50] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. MpNet: Masked and permuted pre-training for language understanding. *Advances in Neural Information Processing Systems* 33 (2020), 16857–16867.
- [51] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [52] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. 2022. Transformer memory as a differentiable search index. *Advances in Neural Information Processing Systems* 35 (2022), 21831–21843.
- [53] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663* (2021).
- [54] Poonam B Thorat, Rajeshwari M Goudar, and Sunita Barve. 2015. Survey on collaborative filtering, content-based filtering and hybrid recommendation system. *International Journal of Computer Applications* 110, 4 (2015), 31–36.
- [55] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
- [56] Jesse Vig, Shilad Sen, and John Riedl. 2012. The tag genome: Encoding community knowledge to support novel interaction. *ACM Transactions on Interactive Intelligent Systems (TuiS)* 2, 3 (2012), 1–44.
- [57] Ashwin K Vijayakumar, Michael Cogswell, Ramprasad R Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. 2016. Diverse beam search: Decoding diverse solutions from neural sequence models. *arXiv preprint arXiv:1610.02424* (2016).
- [58] Jian Wang, Yi Zhang, and Tao Chen. 2012. Unified recommendation and search in e-commerce. In *Information Retrieval Technology: 8th Asia Information Retrieval Societies Conference, AIRS 2012, Tianjin, China, December 17–19, 2012. Proceedings 8*. Springer, 296–305.
- [59] Wenjie Wang, Yiyang Xu, Fuli Feng, Xinyu Lin, Xiangnan He, and Tat-Seng Chua. 2023. Diffusion recommender model. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 832–841.
- [60] Xiaolei Wang, Xinyu Tang, Wayne Xin Zhao, Jingyuan Wang, and Ji-Rong Wen. 2023. Rethinking the evaluation for conversational recommendation in the era of large language models. *arXiv preprint arXiv:2305.13112* (2023).
- [61] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, et al. 2022. A neural corpus indexer for document retrieval. *Advances in Neural Information Processing Systems* 35 (2022), 25600–25614.
- [62] Zihan Wang, Yujia Zhou, Yiteng Tu, and Zhicheng Dou. 2023. NOVO: Learnable and Interpretable Document Identifiers for Model-Based IR. (2023).
- [63] Chuhan Wu, Fangzhao Wu, Tao Qi, and Yongfeng Huang. 2021. Empowering news recommendation with pre-trained language models. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 1652–1656.
- [64] Jun Xu, Xiangnan He, and Hang Li. 2018. Deep learning for matching in search and recommendation. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 1365–1368.
- [65] Tianchi Yang, Minghui Song, Zihan Zhang, Haizhen Huang, Weiwei Deng, Feng Sun, and Qi Zhang. 2023. Auto Search Indexer for End-to-End Document Retrieval. *arXiv preprint arXiv:2310.12455* (2023).
- [66] Jing Yao, Zhicheng Dou, Ruobing Xie, Yanxiong Lu, Zhiping Wang, and Ji-Rong Wen. 2021. USER: A unified information search and recommendation model based on integrated behavior sequence. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 2373–2382.
- [67] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. 2019. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems*. 269–277.
- [68] Kai Yu, Anton Schwaighofer, Volker Tresp, Wei-Ying Ma, and HongJiang Zhang. 2012. Collaborative ensemble learning: Combining collaborative and content-based information filtering via hierarchical Bayes. *arXiv preprint arXiv:1212.2508* (2012).
- [69] Hamed Zamani and W Bruce Croft. 2018. Joint modeling and optimization of search and recommendation. *arXiv preprint arXiv:1807.05631* (2018).
- [70] Hamed Zamani and W Bruce Croft. 2020. Learning a joint search and recommendation model from user-item interactions. In *Proceedings of the 13th international conference on web search and data mining*. 717–725.
- [71] Hamed Zamani, Markus Schedl, Paul Lamere, and Ching-Wei Chen. 2019. An analysis of approaches taken in the acm recsys challenge 2018 for automatic music playlist continuation. *ACM Transactions on Intelligent Systems and Technology (TIST)* 10, 5 (2019), 1–21.
- [72] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. 2023. Scalable and Effective Generative Information Retrieval. *arXiv preprint arXiv:2311.09134* (2023).
- [73] Hansi Zeng, Chen Luo, and Hamed Zamani. 2024. Planning Ahead in Generative Retrieval: Guiding Autoregressive Generation through Simultaneous Decoding. *arXiv:2404.14600* [cs.IR].
- [74] Hailin Zhang, Yujing Wang, Qi Chen, Ruiheng Chang, Ting Zhang, Ziming Miao, Yingyan Hou, Yang Ding, Xupeng Miao, Haonan Wang, et al. 2023. Model-enhanced Vector Index. *arXiv preprint arXiv:2309.13335* (2023).
- [75] Kai Zhao, Yukun Zheng, Tao Zhuang, Xiang Li, and Xiaoyi Zeng. 2022. Joint learning of e-commerce search and recommendation with a unified graph neural network. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1461–1469.
- [76] Wayne Xin Zhao, Yupeng Hou, Xingyu Pan, Chen Yang, Zeyu Zhang, Zihan Lin, Jingsen Zhang, Shuqing Bian, Jiakai Tang, Wenqi Sun, Yushuo Chen, Lanling Xu, Gaowei Zhang, Zhen Tian, Changxin Tian, Shanlei Mu, Xinyan Fan, Xu Chen, and Ji-Rong Wen. 2022. RecBole 2.0: Towards a More Up-to-Date Recommendation Library. In *CIKM*. ACM, 4722–4726.